Build the berth

Purpose, evidence and mission contract

SALTLINE is a single-player industrial-port FPS. Cut three uplink relays, survive the responding machines, and reach the boat. This manual describes the HTML reference, not the current Rust port.

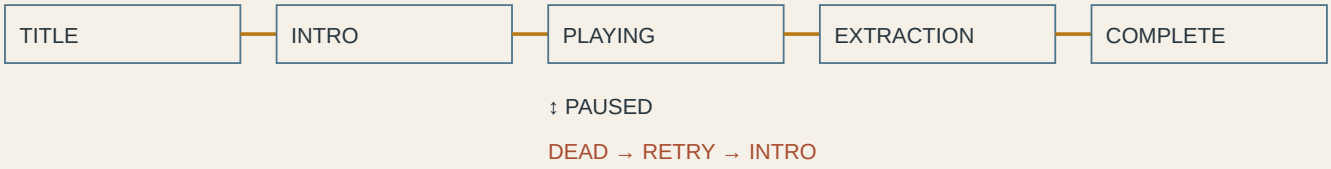


Figure 1: Mission sequence. Extraction is a condition inside PLAYING, not a separate source state.

How to read this manual

Exact facts come from extracted definitions or inspected source branches. **Schematic** plates explain shape, placement or timing and deliberately omit detail. **Interpretation** means implementation advice, not an original game rule.

Use the PDF to understand intent. Use `gallery/assets/*.js` for executable asset construction and `gallery/shared/reference.js` for original geometry and material dependencies. JSON contains derived facts and rendering reports, not model geometry. Resolve gameplay uncertainties against `references/index.html`.

Mission contract

Deploy at the gate. All three relays begin live and may be cut in any order. Hold interaction while near and facing a live relay. Cutting a relay schedules reinforcements. After the third cut, markers change to extraction and pursuit continues.

Entering extraction while alive and after all cuts completes the run. Being killed opens the death flow; retry resets the mission and increments retries. Pause suspends gameplay; losing pointer lock after acquisition or hiding the page also pauses.

Source: mM · line 6182 · search `function mM(`

Quantitative rules

Parameter	Source value
introSeconds	3.5
retryIntroSeconds	1.3
interactRange	2.2
interactFacing	0.45
cutSeconds	2.4
maxAlive	9
spawnStagger	0.85
spawnFar	18

Distances use world metres; time uses seconds. Facing is a dot-product threshold. Relay selection also rejects vertical separation above 2.5 m; very close targets bypass the facing test.

Completion and scoring

Extraction requires horizontal distance below 2.5 m and vertical difference below 3.5 m. A run starts at full health and resets weapons, effects, combat counters and relay states.

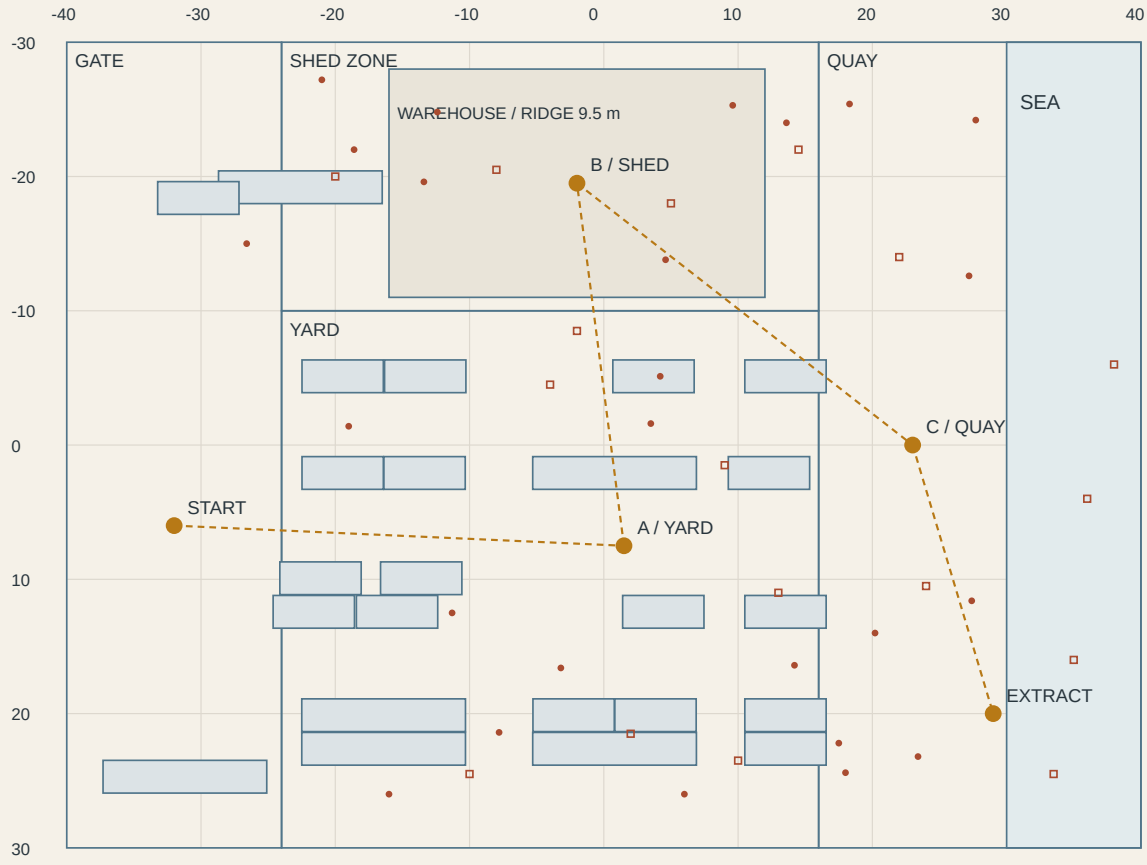
The debrief combines time, accuracy and retries: time below 270 s earns two points, below 400 s one; accuracy at least 50% earns two, at least 30% one; zero retries adds one. Ratings span Messy, Loud, Steady, Clean, Sharp and Surgical.

Reconstruction priorities

Preserve traversable scale, target readability, enemy warning time, weapon cadence and mission pacing. Small bevels, fastener counts and cosmetic noise can be interpreted after those contracts work.

The playable yard

Coordinate map / mission locations / reference footprints



X → east / Z ↓ south / metres / Y is elevation

● Ground spawn □ Air spawn ● Amber: mission location

Container rectangles: first-tier footprints; stack heights, rotated stacks and props omitted.

Dashed connections explain mission flow; they are not verified walkable routes.

Figure 2: Top-down placement guide. Spawn dots are authored candidates; runtime navigation repair and jitter may move ground spawns.

Coordinates and scale

Y is vertical. Map X increases right and Z down. The player camera starts facing −Z before yaw; the robot builders use +Z as forward. Preserve that distinction when converting orientations.

The warehouse spans X −16…12 and Z −28…−11; floor Y 0.03, eaves 7 m and ridge 9.5 m. The quay edge is X 30. The ladder and boat extraction align with Z 20. Source: C1 · line 5442 · search `function C1(`

Mission positions

Location	X / Y / Z
player	−32 / 0 / 6
relay_a	1.5 / 0 / 7.5
relay_b	−2 / 0.03 / −19.5
relay_c	23 / 0 / 0
extraction	29 / 0 / 20

The drawn links express sequence only. Players can choose relay order; containers and walls determine the actual routes.

Space, cover & collision

Level assembly / navigation / encounter placement

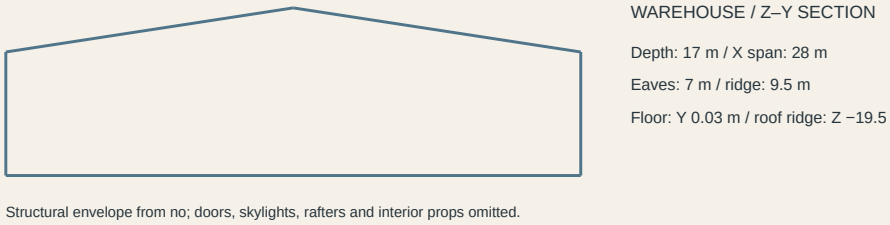


Figure 3: Source-dimension warehouse section; structural envelope only.

Build the space in layers

Start with the ground plane, warehouse shell, quay and world boundaries. Add container stacks, stairs, catwalks and major cover. Then place relays and validate access before adding dressing such as pallets, cables, bollards and decals.

The drawing uses first-tier container sizes from the prop registry. The HTML also stacks, rotates and offsets containers. Preserve those relationships by inspecting `hR` and the `stack` helper in `ob`, rather than treating every catalog bound as final geometry.

Authored zones

Zone	X range	Z range
Gate	−40…−24	−30…30
Yard	−24…16	−10…30
Shed	−24…16	−30…−10
Quay	16…40	−30…30

Zone rectangles classify gameplay space. They are not complete physical boundary walls, and the shed zone is larger than the warehouse footprint.

Collision contracts

The source uses box colliders, capsule movement, raycasts, ground-height queries and a navigation grid. Collider flags distinguish navigation exclusions, bullet-only geometry and player-only blockers. Keep these meanings separate when rebuilding collision in an engine.

Raycasts serve shooting and sensing; ground queries serve foot placement and movement. Visible geometry and collision need not have identical detail. Thin railings, decals and cables should not accidentally seal a walking route. Source: d3 · line 5442 · search `function d3(`

Vertical routes

A yard vantage includes stair-foot location (−2.2, 0, 12.4) and roof location (13.5, 2.59, 12.4). The catwalk connects elevated cover near the container row. Verify stair approach and overhead clearance with the player capsule.

The quay contains a ladder opening and a boat destination. The reference can rescue the player from water; a reconstruction should inspect the player update before choosing fall or water behavior.

Spawn selection

The source supplies 42 candidate spawns. Ground spawns use the navigation grid; air spawns retain altitude. Candidates are scored for distance, concealment and recent use. A preferred spawn distance of 18 m is a scoring preference rather than an absolute rejection boundary.

Recent positions are penalized for six seconds. Spawn jitter is up to ±1.2 m in X/Z and ground jitter requires walkability. Candidate repair searches neighboring walkable cells before gameplay.

Cover and sight

`c1` defines explicit cover points. Hounds navigate toward firing positions; gulls move in three dimensions. Validate sight lines from player eye height and enemy sensors, not only from ground-plane points.

Implementation acceptance: every relay has an accessible approach, all starting paths fit the capsule, the elevated route is usable, and extraction remains reachable after reinforcements appear. Source: hR · line 5442 · search `function hR(` Source: ab · line 5442 · search `function ab(` Source: cb · line 5442 · search `function cb(`

Player & camera

Movement tuning / bodily feedback / interaction

Movement contract

The player is a capsule with radius 0.32 m and standing height 1.8 m. Standing eye height is 1.68 m; crouched eye height is 1.05 m. Keep collision height and camera height distinct.

Parameter	Source value
walk	3.8
sprint	6.4
crouch	1.9
ads	2.4
accelGround	34
decelGround	26
accelAir	9
gravity	21
jumpApex	1
coyote	0.12
jumpBuffer	0.13
crouchHeight	1.15

Speeds are m/s; accelerations and gravity are m/s². Jump apex is 1 m; launch velocity is derived from gravity and apex. Coyote time and input buffering make jumps tolerant around ledges and landing.

Camera contract

World FOV defaults to 74 degrees, with a separate 54-degree viewmodel camera. Sprint adds 6 degrees; aimed FOV targets 52 degrees. Pitch is limited to ±1.5 radians.

Aimed look scales by 0.6. Camera movement includes stride-based bob, strafe roll, landing dip, recoil recovery and damage shake. These are presentation effects; avoid letting them unpredictably move the gameplay collision shape. Source: Pi · line 4180 · search `Pi={` Source: Mt · line 5928 · search `Mt={`

Health and damage

The player starts at 100 health. Damage resets the regeneration timer; regeneration begins after 4 s and restores 25 health/s. Death occurs at zero. Hits trigger directional feedback and camera shake.

Low health causes visual feedback and a one-time radio line below 30 health. Provide cover intervals long enough for the regeneration rule to matter.

Relay hold interaction

A live relay must be within 2.2 m, within vertical tolerance and sufficiently in front. Holding interaction advances normalized progress by elapsed time divided by 2.4. Completing it permanently cuts that relay for the run.

Leaving the interaction or releasing input decays accumulated progress at 1.6 normalized units/s. It does not instantly reset the internal progress to zero. The relay's visual state returns to live while interrupted.

Input and pause

Support movement, look, fire, aim, sprint, crouch, jump, reload, interaction and pause as actions. The reference has keyboard/mouse, gamepad and touch paths, with pointer-lock and drag-look modes. Rebinding is an implementation choice, not a claim about the reference UI.

Acceptance: walking the same path at different frame rates produces consistent speed; the player cannot stand into a ceiling; interrupted interaction decays; pause stops gameplay and resume restores input. Source: `x_` · line 5928 · search `function x_(` Source: mM · line 6182 · search `function mM(`

PAC-9 weapon system

Active viewmodel / firing / reload / impact pipeline



Figure 4: PAC-9 side view rendered from gallery/assets/pac9.js. Hands and transient effects are excluded.

Use the live implementation

The active `dm` builder creates the PAC-9 viewmodel. The fallback weapon stub identifies itself as KR-7 and has different reserve ammunition; that stub is not the weapon specification.

Rule	Value
Magazine / Gv	30 rounds
Reserve / `B_`	150 rounds
Cadence / `O_`	720 rpm
Shot damage / `H_`	26
Range / `V_`	300 m
Tactical reload / `W_`	2.1 s
Empty reload / `G_`	2.72 s

At 720 rpm, the nominal shot interval is 0.0833 s. Treat timer accumulation carefully so low frame rates do not silently reduce cadence.

Visual assembly

Use separate receiver, barrel/handguard, grip, magazine, bolt and optic parts. The reference separates anodized metal, phosphate metal, polymer, tan polymer, brass, glove fabric and rubber. Hands follow the weapon pose with dedicated finger and thumb geometry.

The viewmodel uses its own camera and suppresses world shadow casting. Aim alignment moves the weapon to the sight line rather than simply zooming the world camera.

Fire pipeline

Input and cooldown decide whether a shot can occur. The weapon consumes ammunition, creates a shot direction including spread, and sends it through combat. Combat resolves enemy or world hits, then drives hitmarkers, effects and sound.

The weapon also animates recoil, bolt travel, muzzle flash, shell ejection, heat and smoke. Inspect `Be` inside `dm` for actual shot production; inspect `zv` for collision and damage resolution. Muzzle visuals and the gameplay ray have different roles. Source: dm · line 6054 · search `function dm(`

Reload timeline

A reload tracks normalized time, empty/tactical status and phase. The source drives magazine removal, replacement and bolt actions from this sequence. Reload audio should follow those milestones, not a single unchanging sound.

The precise phase thresholds, spread curves and recoil springs remain in the source. They are not approximated as exact values here: search `function Fe()` and `function Xe(` inside `dm`. Complete weapon tuning can be extended into the data package without changing the page layout.

Feedback priorities

A hit must be distinguishable from a miss; a kill must be distinguishable from an ordinary hit. Surface impacts vary between metal, concrete, wood, water and glass. Reserve and magazine values must remain synchronized with the HUD.

Implementation scenarios

Verify tactical reload with a partly full magazine, empty reload with reserve available, reserve exhaustion, dry firing, holding fire, aim transitions, sprint interruption, pause during reload and retry reset. Confirm that cosmetic recoil and muzzle flash do not introduce duplicate damage.

Interpretation: use a single authoritative shot result to drive damage, sound, particles and UI. This preserves consistent feedback even if an engine uses different rendering and animation APIs. Source: Gv · line 6054 · search `Gv=30` Source: W_ · line 6054 · search `W_=2.1`

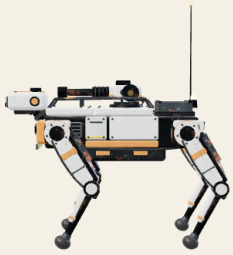
Hound / construction

H-03 / narrow chassis / optical head / dorsal gun



Figure 5: Original hound geometry and procedural surfaces, rendered from gallery/assets/hound.js. Static foot placement uses the original IK solver.





Proportional anchors

Parameter	Source value
stand	0.72
L1	0.4
L2	0.42
l0	0.075
hipX	0.165
hipY	−0.01
hipZ	0.4
footR	0.046

Main shell deck width is 0.35 m; the metal body is 0.27 × 0.20 × 0.98 m. The head envelope is 0.19 × 0.118 × 0.25 m. The long antenna and projecting gun are important silhouette cues.

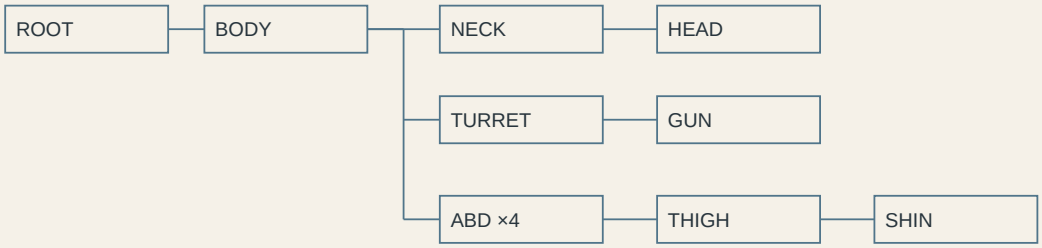
Assembly reading

The extracted JavaScript builds the original skinned geometry and its joint hierarchy. White shell panels sit over dark structural metal, with orange bands and bright optics. Small screws, handles, fins and cables give scale but are secondary to chassis width, leg spacing and head position.

Edit `gallery/assets/hound.js` to change parts and local transforms. The gallery and PNG renderer execute that file directly; no JSON geometry conversion sits between source and image. Shared primitive and skinning helpers remain in `gallery/shared/reference.js`. Source: `Z_` · line 6054 · search `function Z_(`

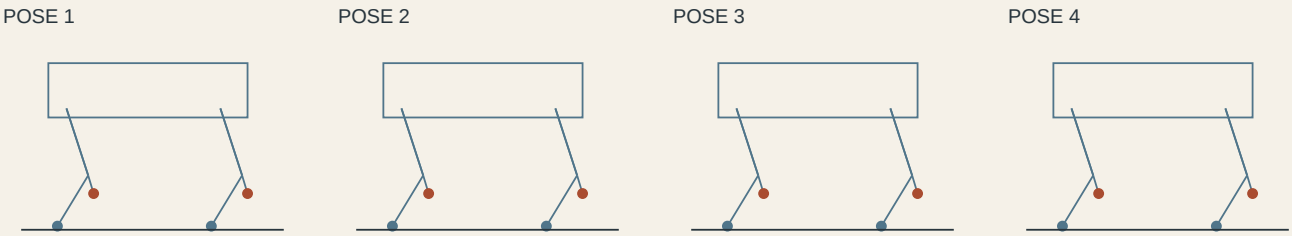
Hound / motion & threat

Rig relationships / planted feet / attack communication



Abduction Z • thigh/knee X • turret Y • gun X • separate antenna and beacon children

Figure 6: Logical articulation tree; four legs repeat the abduction → thigh → shin chain.



Schematic diagonal stepping: FL + RR alternate with FR + RL. Swing arcs lift; stance feet stay planted.

Figure 7: Movement explanation, not extracted animation keyframes. The actual rig solves joints from world-space foot targets.

Grounded movement

The reference alternates diagonal pairs FL+RR and FR+RL. Stance feet retain world positions. Swing targets anticipate movement and turning; a smooth interpolation moves the foot while a sine arc lifts it above the ground. Ground-height queries set landing height.

Inverse kinematics solves hip abduction, thigh and knee angles. Body height, pitch and roll respond to the four foot heights, acceleration and turning. Springs soften those responses. The head aims independently, the turret yaws, the gun pitches and the antenna sways.

This is substantially richer than swinging every limb with a sine wave. Preserve planted feet before adding secondary motion. Source: iM · line 6054 · search `function iM(`

Threat values

Parameter	Source value
hp	140
shotDamage	5
burst	4
burstGap	0.11
telegraph	0.5
runSpeed	4.5
patrolSpeed	1.5
lungeRange	3

Parameter	Source value
lungeDamage	15
lungeCooldown	3.5
lungeSpeed	9.5

Behavior and warnings

Hounds patrol, investigate, search and engage. Their combat sequence includes telegraph, burst fire, cooldown and close-range lunge. Head and lens feedback make the attack legible. Weapon aim is not the same as chassis heading.

Damaged hounds smoke below a health threshold. Death changes body posture, releases attack ownership, slides feet and creates a wreck. Source code, not the rest pose, determines wreck animation.

Gull & encounter pacing

Airborne enemy / shared senses / relay reinforcement



Figure 8: Original gull geometry rendered from gallery/assets/gull.js. Motion-blur discs and the searchlight cone are omitted for this static view.

Gull construction

Four rotor centers are arranged at X ±0.33 and Z ±0.30 m. The body carries rotor supports and protective rings, landing skids, navigation strobes, an underslung sensor gimbal, lamp and gun. Rotor motion and transparent rotor discs serve different visual roles.

The lamp/sensor ball pitches below a yaw pivot. The gun pitches separately. Keep the sensor line of sight and muzzle position tied to their actual transforms.

Parameter	Source value
hp	45
boltDamage	7
boltSpeed	70
burst	3
burstGap	0.12
telegraph	0.55
rangeMin	12
rangeMax	25
altMin	4
altMax	8
maxSpeed	8
fireRange	42

Flight and destruction

Gulls orbit during engagement, investigate last-known positions and avoid obstructions with directional feelers. They check clearance above and below, steer with acceleration and bank in response to forces. They do not follow the hound navigation grid.

At death, a rotor is lost, gravity and spin take over, smoke appears, and collision ends in an explosion. Explosion radius is 3 m; damage is 30. Source: Q_ · line 6054 · search `function Q_(`

Shared AI contracts

The enemy manager coordinates sight, hearing, exposure and attack tokens. Preserve telegraph windows and limits on concurrent attacks; enemy count alone does not determine difficulty.

Search `requestToken`, `think`, `rally`, `provoke` and `releaseToken` in `Em`. Attack ownership must be released on cancellation, death or deactivation. Do not let pooled inactive enemies retain audio loops or tokens. Source: Em · line 6054 · search `function Em(`

Reinforcement schedule

Trigger	Gulls	Hounds
Initial patrol	2	1
Cut 1	3	1
Cut 2	3	2
Cut 3	4	3

Waves depend on cut count, not relay identity. The first scheduled spawn is 1.6 s after the cut; subsequent entries are staggered by 0.85 s. The active cap is 9 queued entries defer when capped.

After all cuts, pursuit starts with a 12 s timer and up to 5 additional enemies. The nominal pursuit interval is 7 s, gated by queue and alive-count conditions. The source also periodically alerts survivors toward the player.

Verification scenarios

Observe a gull under a roof, a hound behind cover, simultaneous attack requests, a wave at the alive cap, death during a warning, loss of sight and a retry while wrecks remain. In every case, warning, attack, damage and sound should agree. Source: mM · line 6182 · search `function mM(`

Surface & atmosphere

Materials / weather / rendering / quality tiers

Material vocabulary

The reference creates procedural albedo, normal and packed surface textures. Named materials provide painted steel, concrete, asphalt, brick, rubber, polymer, fabric, wood, water and glass. Decals add labels, hazards, grime, rust runs, bullet marks and scorch.

Hound geometry groups into shell, metal, glow and lens slots. Its vertex colors include values above one, so these are not plain display-sRGB paint chips. Preserve the intended light/dark hierarchy rather than interpreting raw triples as final screen colors.

Family	Visual intent
Shell	Light painted armor; orange bands
Metal	Dark structure; bright machined fasteners
Rubber	Near-black hoses and feet
Lens / glow	Amber idle; redder attack feedback
Ground	Wet asphalt and concrete; puddle variation
Sea	Dark moving surface with horizon haze

Readability in weather

Wetness changes roughness and surface response. Rain includes falling streaks and splashes; interior bounds suppress outdoor effects. Puddles and the sea use different scales. Do not apply kilometre-scale sea texture tiling to small puddles. Source: Gg · line 5442 · search `function Gg(` Source: O1 · line 5814 · search `function O1(`

Lighting and post-processing

The renderer includes sky and sun lighting, shadows, fog/volumetrics, ambient occlusion, bloom, adaptation, lens dirt, streaks, flare, vignette, grain and sharpening. These effects support the industrial atmosphere but must leave relay lights and attack cues readable.

Damage, low health, aiming, flash, fade and saturation are exposed as post-process signals. Drive them from gameplay events rather than mixing game rules into a shader.

Quality tiers

Tier	Scale	Shadow
low	0.7	2048
medium	0.85	2048
high	1	4096
ultra	1	4096

The complete tier definitions are in the data package, including MSAA, texture size, volumetric steps, AO, planar reflections, bloom, motion blur and rain. Runtime adaptive scaling can modify the effective resolution.

Reconstruction guidance

Build a readable daylight scene first, then add sky/fog and wet surfaces. Validate combat cues before enabling glare and grain. Quality reductions should change visual cost while preserving collision, sensing, interaction reach and weapon behavior.

Acceptance: optics remain visible against both warehouse shadow and bright sky; rain does not visually continue through the warehouse; texture repetition remains plausible at player scale.

Asset thumbnails / 1

Executable JavaScript builders / original procedural surfaces / studio lighting



container20
6.07 × 2.59 × 2.45 m / nominal
assets/containers.js



container40
12.2 × 2.59 × 2.44 m / nominal
assets/containers.js



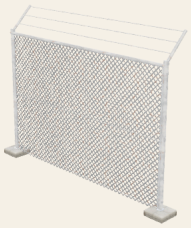
container40hc
12.2 × 2.9 × 2.44 m / nominal
assets/containers.js



jersey barrier
3 × 0.86 × 0.61 m / nominal
assets/jersey_barrier.js



light mast
0.99 × 14 × 1.9 m / nominal
assets/light_mast.js



fence panel
3.32 × 2.4 × 0.53 m / nominal
assets/fence_panel.js



guard booth
3.11 × 2.55 × 2.77 m / nominal
assets/guard_booth.js



gate arm
5.34 × 1.19 × 0.46 m / nominal
assets/gate_arm.js



bollard mooring
0.74 × 0.61 × 0.74 m / nominal
assets/bollard_mooring.js

Source of truth

Every tile is a real Three.js render of its named builder. Edit the listed file under `gallery/`, then rebuild to refresh both the gallery and PDF. Nominal dimensions are read from the extracted JavaScript registry.

Assets are framed individually, so image sizes are not mutually to scale. Lighting and exposure are standardized; construction geometry and surfaces come from the extracted source. Detailed colliders remain in the builders.

Reading the views

Asset thumbnails / 2

Executable JavaScript builders / original procedural surfaces / studio lighting



bollard post
0.17 × 1 × 0.18 m / nominal
assets/bollard_post.js



relay node
2.57 × 3.18 × 1.17 m / nominal
assets/relay_node.js



shelf rack
2.75 × 4.2 × 1.13 m / nominal
assets/shelf_rack.js



stairs metal
3.12 × 3.62 × 1.08 m / nominal
assets/stairs_metal.js



catwalk
3 × 1.25 × 1.22 m / nominal
assets/catwalk.js



reefer unit
0.63 × 2.3 × 2.3 m / nominal
assets/reefer_unit.js



barrel
0.59 × 0.9 × 0.59 m / nominal
assets/barrel.js



pallet
1.21 × 0.14 × 1.01 m / nominal
assets/pallet.js



pallet stack
1.26 × 1.15 × 1.05 m / nominal
assets/pallet_stack.js

Source of truth

Every tile is a real Three.js render of its named builder. Edit the listed file under `gallery/`, then rebuild to refresh both the gallery and PDF. Nominal dimensions are read from the extracted JavaScript registry.

Assets are framed individually, so image sizes are not mutually to scale. Lighting and exposure are standardized; construction geometry and surfaces come from the extracted source. Detailed colliders remain in the builders.

Reading the views

Asset thumbnails / 3

Executable JavaScript builders / original procedural surfaces / studio lighting



crate wood
1.11 × 1 × 0.91 m / nominal
assets/crate_wood.js



crate stack
1.21 × 1.91 × 1.04 m / nominal
assets/crate_stack.js



ibc tote
1.2 × 1.14 × 1 m / nominal
assets/ibc_tote.js



forklift
3.41 × 2.19 × 1.25 m / nominal
assets/forklift.js



cable reel
1.3 × 1.3 × 0.86 m / nominal
assets/cable_reel.js



tire stack
0.81 × 0.99 × 0.81 m / nominal
assets/tire_stack.js



dumpster
1.92 × 1.21 × 1.22 m / nominal
assets/dumpster.js



generator
2.22 × 1.41 × 1.04 m / nominal
assets/generator.js



jerry can
0.35 × 0.45 × 0.17 m / nominal
assets/jerry_can.js

Source of truth

Every tile is a real Three.js render of its named builder. Edit the listed file under `gallery/`, then rebuild to refresh both the gallery and PDF. Nominal dimensions are read from the extracted JavaScript registry.

Assets are framed individually, so image sizes are not mutually to scale. Lighting and exposure are standardized; construction geometry and surfaces come from the extracted source. Detailed colliders remain in the builders.

Reading the views

Asset thumbnails / 4

Executable JavaScript builders / original procedural surfaces / studio lighting



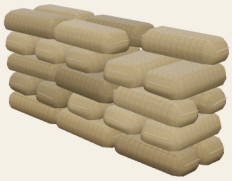
traffic cone
0.36 × 0.7 × 0.36 m / nominal
assets/traffic_cone.js



gas cylinders
0.91 × 1.6 × 0.5 m / nominal
assets/gas_cylinders.js



tarp cargo
2.41 × 1.46 × 1.48 m / nominal
assets/tarp_cargo.js



sandbags
1.99 × 0.9 × 0.62 m / nominal
assets/sandbags.js



toolbox cart
0.86 × 0.96 × 0.54 m / nominal
assets/toolbox_cart.js



floodlight stand
0.98 × 2.24 × 1.65 m / nominal
assets/floodlight_stand.js



cardboard boxes
0.98 × 0.74 × 0.92 m / nominal
assets/cardboard_boxes.js



pipe stack
5.96 × 0.84 × 1.2 m / nominal
assets/pipe_stack.js



sign post
0.6 × 2.41 × 0.3 m / nominal
assets/sign_post.js

Source of truth

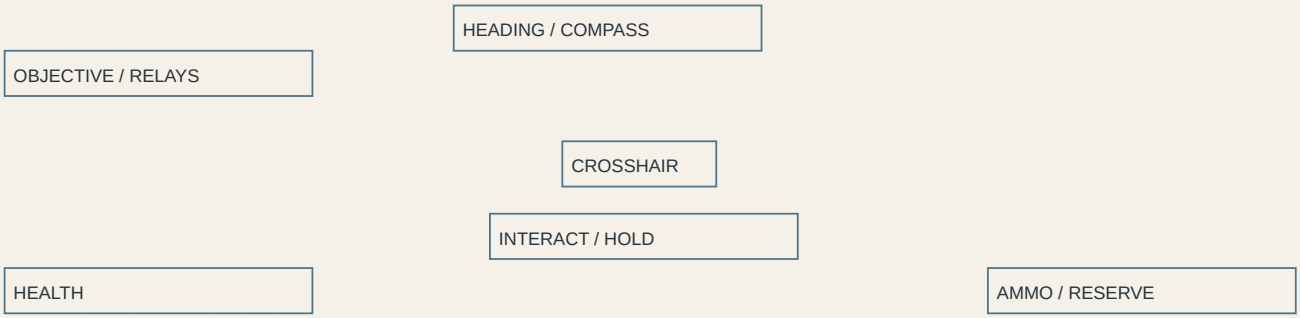
Every tile is a real Three.js render of its named builder. Edit the listed file under `gallery/`, then rebuild to refresh both the gallery and PDF. Nominal dimensions are read from the extracted JavaScript registry.

Assets are framed individually, so image sizes are not mutually to scale. Lighting and exposure are standardized; construction geometry and surfaces come from the extracted source. Detailed colliders remain in the builders.

Reading the views

HUD, audio & delivery

Presentation contracts / build instructions / implementation checks



Layout schematic; normalized to the reference HUD hierarchy, not pixel-exact.

Figure 9: Screen-space hierarchy of the reference HUD. Exact spacing adapts to display and input mode.

HUD and menus

Display heading, relay objective/count, world markers, crosshair, hitmarkers, directional damage, interaction progress, radio messages, health, magazine, reserve and reload cues. Pause, death and completion have distinct screens.

Settings include quality, sensitivity, invert-Y, volume and FOV. Touch mode changes layout and supplies stick/fire/aim/jump/reload/interact/pause controls. Keep control prompts synchronized with the active input mode. Source: hM · line 6076 · search `function hM(` Source: pM · line 6099 · search `function pM(`

Sound event map

Event	Sound family
Player fires	Rifle shot and interior/outdoor tail
Reload phase	Magazine out / in / bolt
Footstep	Concrete / water / metal
Enemy	Rotor / servo loops, warning, firing, death
World	Rain, sea, wind, thunder, foghorn
Mission	Relay cut, objective, completion, death

The source synthesizes audio buffers and spatializes world sounds. Music intensity changes from quiet patrol to contact and active waves/extraction. Interior transitions affect ambience and reverb. Pause and pooled-enemy cleanup must stop or attenuate the appropriate loops. Source: u_ · line 5928 · search `function u_(`

Source package

Run `python3 docs/game-manual/build.py` from the repository root. Node extracts gameplay facts and renders the editable asset JavaScript in Chromium. Python generates map and behavior diagrams; Typst compiles this PDF. No game code or reference source is modified.

`reference.json` contains the source SHA-256, searchable anchors and line numbers. Asset thumbnails and measured bounds are generated from the asset JavaScript, while gameplay tables are regenerated from the HTML reference. Inspect anchors before carrying numbers into a port.

Verification checklist

Validate movement and capsule clearances; all relay approaches; interruption decay; wave cap and retry cleanup; tactical/empty reload; enemy warning cancellation; grounded feet; air clearance; extraction gating; sound loop retirement; and HUD consistency.

The document build checks extraction counts, SVG XML validity and successful Typst compilation. It does not prove game equivalence. Compare a reconstructed level and enemy behavior against the original in play.

Deliberate interpretation

Thumbnail geometry comes from executable JavaScript builders. Studio lighting, dry surfaces, fixed inspection poses and omission of transient effects are presentation choices. The gait strip explains behavior rather than providing authoritative poses. The source package remains available wherever the drawing leaves implementation freedom.

Source navigation

Evidence index / JavaScript asset authority / regeneration

Reference fingerprint (SHA-256): `f34e6debbfa7526b72be2daa24595c2f195e102a7aede4f22e59138c99cd6852`

Searchable definitions

Symbol	Responsibility
Pi / Mt	Capsule, eye height, camera and movement
Yi / mM	Mission, waves, relay and extraction logic
C1 / hR	Level assembly, spawns and container rows
no / ab	Warehouse dimensions and construction
lb / cb	Quay, ladder and extraction
Gi / Yg	Prop dimensions and registered builders
Z_ / iM	Hound geometry and motion
j_ / Q_	Gull geometry and flight
Em	Enemy manager, shared senses and tokens
dm / zv	PAC-9 and combat ray resolution
Gg / O1	Materials, wet ground, sea and rain
hM / pM	HUD, menus and settings
u_	Audio generation, mixing and spatial loops

Read the executable assets

`gallery/assets/hound.js`, `gull.js` and `pac9.js` hold the extracted original geometry builders. Each prop builder has its own named file; the three container variants share `containers.js`. The original Three.js library, primitive helpers, materials and registry wiring live in `gallery/shared/reference.js`.

`gallery/registry.js` supplies readable names, static poses and material adapters. `gallery/gallery.js` provides camera framing and inspection controls. Those JavaScript files are the editable asset authority. A normal manual build executes them; it does not overwrite them from the original HTML.

The thumbnail manifest records camera view, measured bounds, builder path and builder-file SHA-256. It is a derived report. Neither it nor `reference.json` reconstructs asset geometry.

Units and transforms

World distances are metres, elapsed times seconds, speeds metres/second, rotations radians unless explicitly described as FOV degrees. Material colors are source values rather than guaranteed display colors. A prop registry size is nominal and may differ from its detailed collider shape.

World Y is up. Model forward is +Z; the player camera uses −Z before yaw. Source Euler transforms use XYZ order. Do not mirror the front and rear legs when converting to a different engine convention.

Check the source, then interpret

Use exact values for mechanics and major spatial relationships. Use diagrams for assembly intent. Where cosmetic freedom remains, choose an implementation that preserves silhouette, scale, material contrast and motion readability.

The map omits dressing, stack heights and rotations. The dashed mission links are not a route planner. The walking strip is explanatory. Asset grids show actual rendered geometry, including procedural material detail, at individually normalized scales.

Rebuild and review

A rebuild refreshes gameplay facts, renders the asset JavaScript to PNGs, regenerates explanatory vectors and compiles the manual. Refresh asset source from the HTML only with the explicit `rip-assets.cjs` command; that import step overwrites extracted builder edits. The extractor fails if expected source definitions or location anchors disappear. Review the complete PDF after changing layout or dimensions.

The build does not run the original game or certify gameplay equivalence. Acceptance requires walking the level, cutting relays in multiple orders, exercising both enemies and comparing actual combat feedback.